



Research Intake 2026

Day 35

Diffusion Models & Image Generation

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1	A Second Path to Generation	4
2	The Forward Diffusion Process	5
2.1	The Noise Schedule	5
2.2	The Closed-Form Property: Sampling Any Timestep Directly	6
2.3	Why the Forward Process Is Deliberately Simple	6
3	The Reverse Process & Denoising	7
3.1	Connection to Score-Based Generative Modeling	7
3.2	Why One Model Produces Diverse Outputs	8
4	Training a Diffusion Model	8
4.1	The Simplified Training Objective	9
4.2	Why This Training Stability Is Diffusion's Key Practical Advantage	9
4.3	Training Loop Mechanics	10
5	U-Net as the Denoising Backbone	10
5.1	The Same Encoder-Decoder-With-Skip-Connections Structure	10
5.2	Timestep Embedding: Telling the Network Which Noise Level It Sees	11
5.3	Attention Layers for Long-Range Coherence	11
6	Sampling Strategies & Speed	12
6.1	DDPM's Original Slow Sampling	12
6.2	DDIM: A Faster Deterministic Sampling Shortcut	13
6.3	Further Speedups: Distillation and Few-Step Samplers	13
6.4	The Speed-Quality Tradeoff	14
7	Conditional & Guided Generation	15
7.1	Classifier Guidance and Classifier-Free Guidance	15
7.2	Text-Conditioned Generation and Cross-Attention	15
8	Latent Diffusion	16
8.1	The Motivation: Pixel-Space Diffusion Is Expensive	16
8.2	The Latent Diffusion Insight	17
8.3	Why This Made High-Resolution Diffusion Practical	18
9	GANs vs. Diffusion: A Direct Comparison	18
9.1	Training Stability	19
9.2	Sample Quality and Diversity	19
9.3	Inference Speed	19
9.4	Training Cost and Data Efficiency	20
9.5	Practical Guidance: When Each Is the Better Choice	20
10	Research Frontiers	21
10.1	Consistency Models and Closing the Speed Gap	21
10.2	Video Diffusion: A Forward Reference to Day 36	22
10.3	3D-Aware Diffusion: A Forward Reference to Day 36	22
10.4	Diffusion Beyond Images	23
10.5	Ethical Considerations: An Even More Capable Mechanism	23
11	Common Pitfalls: A Practical Debugging Checklist	24

11.1 A Note on Reproducibility	25
11.2 A Note on Comparing Against Published Benchmarks	25
Glossary of Terms	26
Chapter Summary: Key Takeaways	27

The future of AI is bright, and you can be part of it!

1 A Second Path to Generation

Day 34 §1 opened the module's generative arc and, in its closing sections, promised this day directly: Day 34 §10.1 previewed a comparison between GANs and a second, structurally different generative paradigm, and this day is that paradigm developed in full. Diffusion models solve the same underlying problem Day 34 §1 defined — learn to produce new images that look like they belong to a training distribution, with no per-input ground truth to train against — through a genuinely different mechanism than Day 34's adversarial game between two competing networks.

It is worth situating this day within the module's larger arc one more time before the mechanics begin, since Day 33 §10.4 first flagged this transition and it is easy to lose track of across two full days of generative content. Days 26 through 33 were entirely discriminative: map a given input to a label, box, or mask, with a single objectively correct answer for any given input. Day 34 and this day are both generative: produce new content with no single correct output at all, only a broad notion of plausibility relative to a training distribution. This day is not a new arc, only the second of two structurally different mechanisms within the same generative arc Day 34 opened — a distinction worth keeping explicit, since the temptation to read this day as somehow separate from Day 34 rather than its direct continuation would miss the entire point of Section 9's comparison.

The high-level intuition, before any mechanics: take a real image and gradually destroy its structure by adding random noise over many small steps, until nothing recognizable remains — pure static. Diffusion models then learn to reverse exactly this process, one small denoising step at a time, starting from pure noise and gradually recovering a coherent image. Generation, in this framework, is not a single forward pass producing an image directly (Day 34's generator), but a repeated, iterative refinement — starting from random noise and asking, over and over, "what would this look like with slightly less noise?" until a clean image emerges.

It is worth being explicit about why this approach emerged specifically as a response to Day 34's training difficulties, since diffusion is not simply an alternative that happened to exist alongside GANs from the start. Day 34 §3 and §4 developed, at length, why adversarial training is difficult: two networks chasing a moving target defined by each other, no single trustworthy loss curve, and several well-characterized failure modes (mode collapse, instability, an overpowered discriminator) with no guaranteed fix. Diffusion models were developed substantially in direct response to this instability, replacing Day 34's two-network adversarial game with a single network trained against a fixed, simple, well-behaved target — Section 4 develops exactly what that target is and why it sidesteps Day 34's core training difficulty almost entirely.

It is worth previewing this day's own structure before the mechanics begin, since it mirrors — deliberately — the shape Day 34 used for GANs, making the two days easy to read side by side. Sections 2 through 4 build the core training mechanism (the forward process, the reverse process, and the training objective), directly paralleling Day 34 §2–§3's generator-discriminator framework and loss function. Section 5 covers

the backbone architecture, paralleling Day 34 §5's DCGAN guidelines. Sections 6 through 8 cover practical extensions (speed, conditioning, and a key efficiency insight), paralleling Day 34 §6–§8's conditional, translation, and high-resolution extensions. Section 9 delivers the direct comparison both days have been building toward, and Section 10 closes with research frontiers and ethics, exactly as Day 34 §10 did.

Callout Box: *Figure 1 makes the core idea concrete before any of the mechanics below. Reading left to right: a real image is progressively corrupted with more and more random noise until, at the far right, nothing recognizable remains — pure static. Reading right to left is generation: starting from pure noise, a trained model removes a small amount of noise at each step, gradually revealing a coherent image that was never explicitly specified in advance. The left-to-right direction (Section 2) is fixed and requires no learning at all; the right-to-left direction (Section 3) is the one thing the model actually learns.*

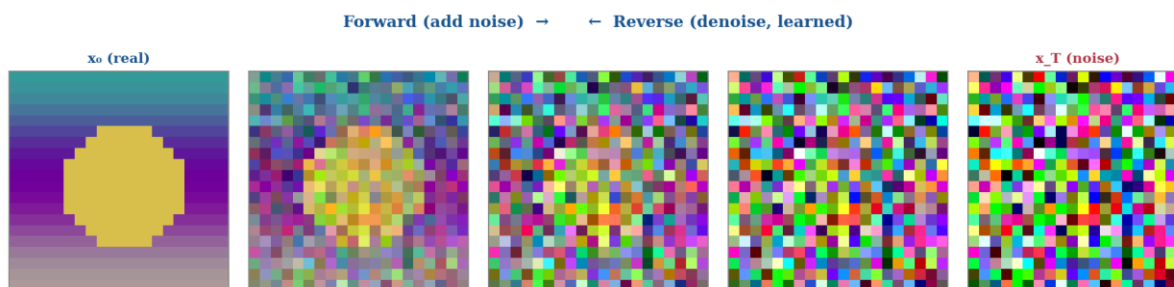


Figure 1. The forward process (left to right) is a fixed, non-learned corruption of a real image into pure noise. Generation runs this in reverse (right to left): a trained model removes noise step by step, starting from randomness and ending at a coherent image.

2 The Forward Diffusion Process

The forward diffusion process, illustrated in Figure 2, takes a real training image x_0 and produces a sequence of increasingly noisy versions $x_1, x_2, \dots, x_T$ by adding a small amount of Gaussian noise at each of T steps, until x_T is indistinguishable from pure random noise. This is formalized as a Markov chain: each step's output depends only on the immediately preceding step, not on the full history of how the image got there — a deliberately simple structure that Section 2.2 will show has a valuable mathematical consequence.

2.1 The Noise Schedule

The amount of noise added at each step is governed by a noise schedule, a predetermined sequence of values controlling how much of the previous step's signal is retained versus how much new noise is mixed in at each of the T steps. A linear schedule increases the noise added at a constant rate across all T steps; a cosine schedule instead adds noise more slowly near the beginning and end of the process and more quickly in the middle, which empirically tends to produce a smoother, more learnable trajectory for the reverse process (Section 3) to model, since it avoids the signal degrading too abruptly at either end of the chain. Choosing this schedule is itself a design decision made once for the whole training process, not something the model learns.

It is worth being concrete about why the schedule's shape actually matters for the reverse process's learnability, rather than treating it as an arbitrary hyperparameter. If too much noise is added too early in the schedule, the model has very little genuine signal left to learn from during the many later timesteps, which are all close to indistinguishable pure noise — wasting a large fraction of the schedule's steps on a regime where the denoising task is nearly impossible to learn anything useful from. A well-chosen schedule spreads the genuinely informative, learnable denoising transitions more evenly across the full range of timesteps, which is precisely the empirical motivation behind the cosine schedule's slower noise addition near the schedule's endpoints.

2.2 The Closed-Form Property: Sampling Any Timestep Directly

Because the forward process's noise is Gaussian and added according to a fixed, known schedule, it has a valuable closed-form property: the noisy image at any timestep t can be computed directly from the original image x_0 and a single sample of random noise, using a formula derived from the schedule, without ever having to actually simulate all of the intermediate steps x_1 through x_{t-1} one by one. This is not a minor implementation convenience; it is what makes Section 4's training procedure practical at all, since it means a single training step can jump directly to a randomly chosen timestep t rather than needing to run the full T -step forward chain for every training example on every training iteration.

Forward Process: a Fixed, Non-Learned Markov Chain

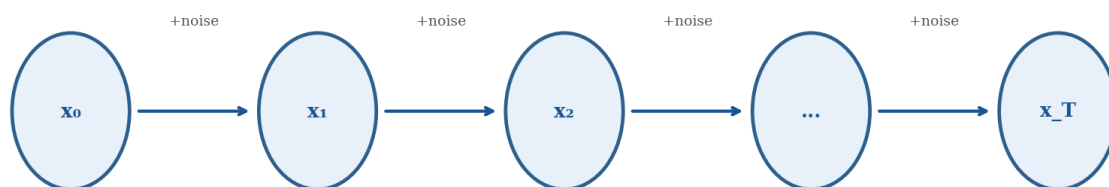


Figure 2. The forward process is a fixed Markov chain, not a learned network — each arrow simply adds a small, predetermined amount of Gaussian noise. Its closed-form property (§2.2) lets training jump directly to any timestep without simulating every intermediate step.

2.3 Why the Forward Process Is Deliberately Simple

It is worth being explicit about a design choice this section has left implicit: the forward process is deliberately not learned at all, in sharp contrast to Day 34's generator, which is the one and only learned component doing all of the generative work in a GAN. Diffusion instead puts all of the model's learning capacity into a single job — the reverse process, Section 3's subject — while the forward process remains a fixed, known, mathematically simple corruption that requires no training whatsoever. This asymmetry (one fixed, simple process; one learned, harder process) is a large part of why diffusion training turns out

to be so much more stable than Day 34's two-network adversarial setup: there is no second network whose behavior the first has to chase, only a single, well-defined target the one learned network is trained to hit.

It is worth restating this asymmetry once more in the plainest possible terms, since it is the single idea this whole section has been building toward and it recurs throughout the rest of this day. A GAN has two moving, learned parts (Day 34 §2's generator and discriminator) whose interaction is precisely what makes training hard to read (Day 34 §3.2). A diffusion model has one fixed, unlearned part (this section's forward process) and one learned part (Section 3's reverse process) whose target — Section 4.1's noise-prediction objective — never shifts underneath it during training. Everything Section 4.2 will say about diffusion's training stability traces back to this one structural choice.

3 The Reverse Process & Denoising

The reverse process, illustrated in Figure 3, is what a diffusion model actually learns: given a noisy image x_t at some timestep, predict a slightly less noisy version — or, in the most common practical formulation, predict the noise component itself that was added to produce x_t from x_{t-1} , which turns out to be a mathematically equivalent and empirically easier quantity for a network to learn than predicting the denoised image directly. Section 4 develops this noise-prediction framing as the actual training objective.

3.1 Connection to Score-Based Generative Modeling

Diffusion models are closely related to a separate line of research called score-based generative modeling, and the connection is worth understanding rather than treating diffusion as an isolated, self-contained idea. The score function of a probability distribution is defined as the gradient of the log-density with respect to the data — intuitively, a vector field pointing, at every point in data space, in the direction that most increases the likelihood of that point under the true data distribution. A model trained to predict noise, as Section 3 describes, turns out to be learning something mathematically equivalent to this score function at each noise level: predicting which direction to move a noisy sample to make it more probable under the data distribution is, up to a known scaling factor, the same computation as predicting the noise that was added to it. This equivalence is why diffusion models and score-based generative models are often discussed as two framings of the same underlying idea rather than as competing techniques.

It is worth grounding this equivalence with an intuitive picture, since the formal statement above can otherwise feel abstract. Imagine the space of all possible images, with real, natural-looking images clustered in certain regions and unlikely, noise-like images spread thinly everywhere else. The score function at any point in this space points "uphill," toward regions of higher image-likelihood. A noisy image, by construction, sits somewhere between a genuine image and pure noise in this space, and predicting the noise that was added to produce it is, in effect, predicting exactly the direction back toward the likely, natural-

image region it came from — which is precisely what following the score function would also tell you to do.

3.2 Why One Model Produces Diverse Outputs

A single trained denoising model can generate an unlimited variety of different outputs, and it is worth being precise about where that diversity actually comes from, since the model itself is deterministic given a fixed input. The diversity comes entirely from Section 2's random noise sampled as the starting point x_T : because generation begins from pure random noise and different random samples of x_T lead the same deterministic denoising process down different trajectories, a single trained model produces different images for different random starting points, exactly as Day 34 §2's generator produced different images for different random noise inputs, but through a repeated, iterative refinement rather than a single forward pass.

It is worth contrasting this diversity mechanism directly against Day 34 §4.1's mode collapse, since the comparison clarifies exactly why diffusion tends not to exhibit an equivalent failure mode. A GAN's generator can, per Day 34 §4.1, learn to map many different noise inputs to the same or nearly the same output, since its only training pressure is fooling the discriminator, with nothing explicitly rewarding using the full range of its noise input space. A diffusion model's noise-prediction objective (Section 4.1) has no equivalent degenerate shortcut available: because training samples real images directly and asks the network to recover them from their own specific noised versions, there is no analogous incentive for the network to collapse many different noise trajectories onto the same output — every training example pulls the network toward correctly reconstructing that example's own specific image, across the full diversity of the training set.

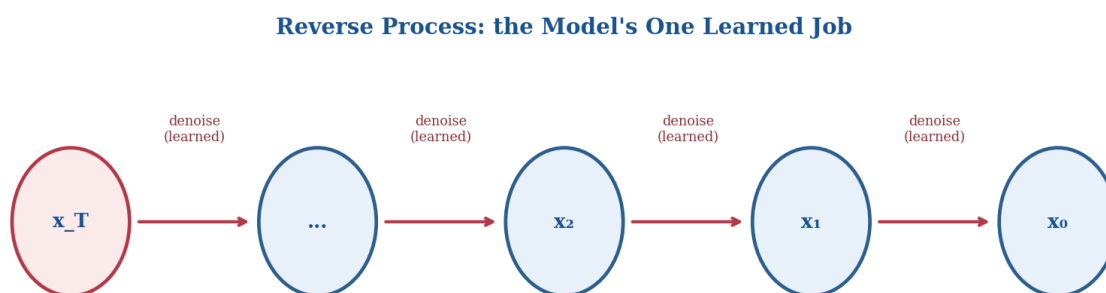


Figure 3. The reverse process is the model's one learned job: starting from pure noise (x_T), repeatedly predict and remove a small amount of noise until a coherent image (x_0) emerges. Every arrow in this diagram represents an application of the same trained network.

4 Training a Diffusion Model

The training objective this section develops is, deliberately, one of the simplest loss functions this compendium has covered — a direct, striking contrast to Day 34 §2–§3's adversarial minimax loss, and worth reading with that contrast in mind throughout.

4.1 The Simplified Training Objective

Training a diffusion model, illustrated in Figure 4, follows a straightforward recipe: take a real training image x_0 , sample a random timestep t , use Section 2.2's closed-form property to directly compute the noisy version x_t at that timestep (along with the specific noise that was added to produce it), pass x_t and t into the network, and train the network to predict that specific noise as accurately as possible. The loss is simply the mean squared error between the network's predicted noise and the actual noise that was added — a standard regression loss, structurally no different from loss functions this compendium has used in far simpler contexts, and a world apart from Day 34 §3's two-network minimax formulation.

It is worth being explicit about why predicting noise specifically, rather than directly predicting the clean image x_0 , turns out to work better in practice, since both are mathematically valid targets given Section 2.2's closed-form relationship between them. Predicting the noise tends to produce a more stable training signal across all timesteps simultaneously: at early timesteps (barely-corrupted images), the clean image and the noisy input are nearly identical, making direct image prediction a nearly trivial, low-information task, while at late timesteps (nearly pure noise), predicting the clean image directly requires the network to somehow recover almost all of the original structure from almost nothing. Noise prediction, by contrast, poses a comparably scaled, well-behaved regression problem at every timestep, which is a significant part of why it became the standard practical choice.

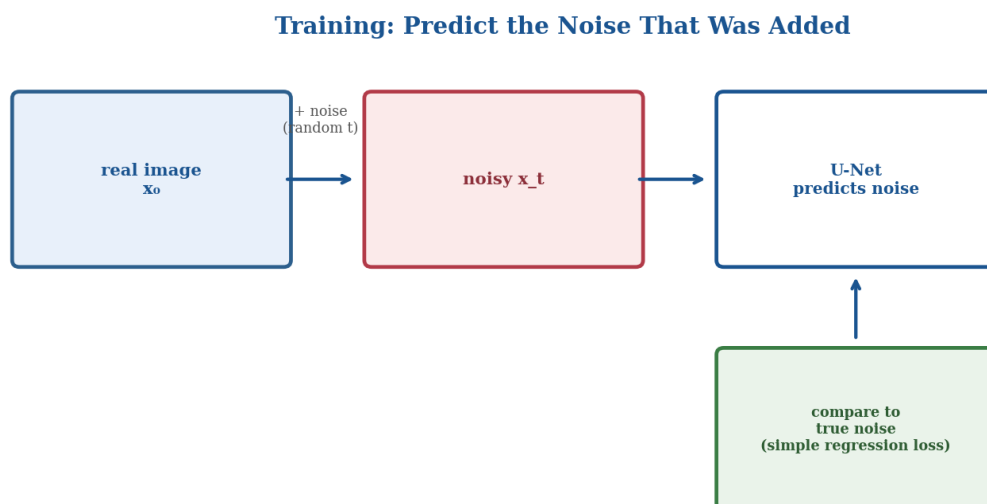


Figure 4. Diffusion training in one picture: take a real image, add a known amount of noise at a random timestep, ask the network to predict that noise, and compare against the true noise with a simple regression loss — no second network, no adversarial game.

4.2 Why This Training Stability Is Diffusion's Key Practical Advantage

It is worth being explicit about exactly what this simplicity buys in practice, connecting directly back to Section 1's framing of diffusion as a response to Day 34's training difficulties. There is no second network to balance against (Day 34 §3.3's balancing problem, simply absent here). There is no adversarial loss whose value fails to indicate progress (Day 34 §3.2's loss-reading problem — a diffusion model's training loss, this simple regression error, is a genuinely reliable, monotonically informative progress signal in a way neither GAN loss curve was). And there is no mode collapse, training instability, or overpowered-discriminator failure mode to diagnose (Day 34 §4's entire catalog of failure modes has no real analogue here, since there is only one network being trained against one fixed, well-behaved target). This is precisely why diffusion models are widely regarded as substantially easier and more predictable to train than GANs, and it is the single most consequential practical difference Section 9's direct comparison will return to.

4.3 Training Loop Mechanics

A single diffusion training step, concretely: sample a batch of real images, sample a random timestep independently for each image in the batch, compute each image's noisy version at its sampled timestep (Section 2.2's closed-form shortcut), run the network forward to predict the noise, compute the mean squared error against the true noise, and backpropagate — a single, standard forward-backward training step with no alternating updates between two networks, no frozen-parameter bookkeeping between generator and discriminator updates (Day 34 §3.1's alternating structure), and no need to track which of two networks is currently ahead.

5 U-Net as the Denoising Backbone

The network that actually performs Section 3 and Section 4's noise-prediction task is, in the overwhelming majority of diffusion implementations, a U-Net — a direct callback to Day 33 §4's segmentation architecture, and worth taking seriously as a genuine architectural reuse rather than a superficial naming coincidence.

5.1 The Same Encoder-Decoder-With-Skip-Connections Structure

Diffusion's denoising U-Net uses exactly the structure Day 33 §4.1 developed: a downsampling encoder path, a mirrored upsampling decoder path, and skip connections carrying fine spatial detail directly from each encoder level to its matching decoder level. The task has changed — predicting per-pixel noise rather than Day 33's per-pixel class label or Day 34 §7.1's translated image — but the underlying architectural need is the same one Day 33 §3.4 identified: a network processing a full-resolution image needs both coarse, large-receptive-field context (to understand what the image broadly depicts) and fine spatial detail (to predict noise accurately at every individual pixel), and skip connections remain the same solution to that

same resolution-versus-context tradeoff regardless of whether the output is a segmentation mask or a predicted noise map.

It is worth being explicit about just how directly reusable Day 33's architecture turns out to be, since it is a genuinely striking instance of this compendium's cross-day architectural reuse. The input and output shapes match exactly (both a full-resolution, multi-channel spatial tensor), the encoder-decoder-with-skip-connections backbone matches exactly, and the loss function (mean squared error against a target map) is arguably simpler than Day 33's own segmentation losses (Day 33 §7.1's class-imbalance concerns, for instance, have no real analogue in a noise-prediction target). The two genuinely new additions — Section 5.2's timestep embedding and Section 5.3's attention layers — are the entire architectural delta between a Day 33 segmentation U-Net and a Day 35 denoising U-Net.

5.2 Timestep Embedding: Telling the Network Which Noise Level It Sees

A segmentation U-Net (Day 33 §4) has no notion of "which stage" it is operating at — it simply maps one input image to one output mask. A denoising U-Net has a genuinely new requirement Day 33's architecture did not need: the same network must handle every timestep from 1 to T , and a noisy image at an early timestep (barely corrupted) requires a very different denoising correction than the same network sees at a late timestep (almost pure noise). The network needs to know which timestep it is currently operating at, and this is provided via a timestep embedding, illustrated in Figure 5: the scalar timestep t is converted into a vector representation (commonly using the same sinusoidal positional encoding scheme Day 30 §7's transformer discussion introduced for token positions) and injected into the U-Net at multiple points throughout its encoder and decoder path, letting every layer's computation be conditioned on which noise level it is currently being asked to remove.

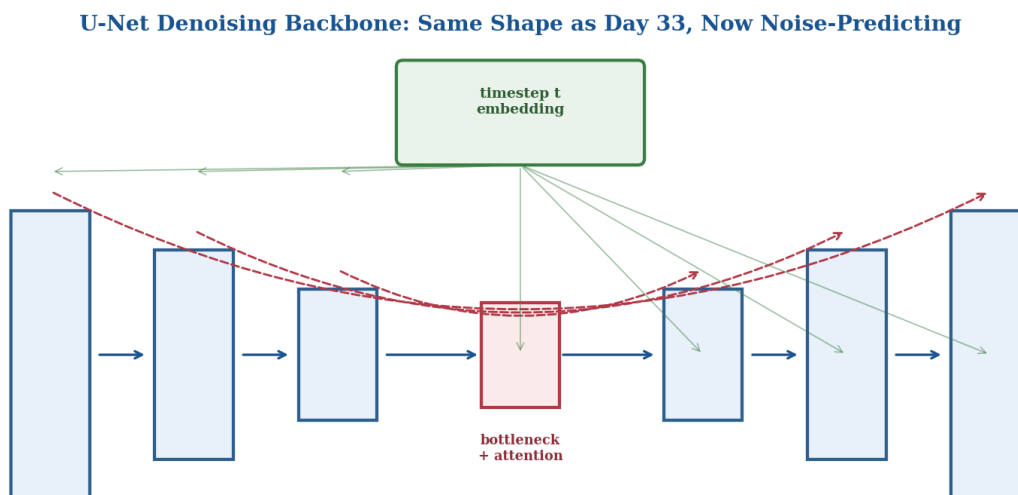


Figure 5. The denoising backbone is Day 33's U-Net with two additions: a timestep embedding injected throughout, telling the network which noise level it is denoising, and attention layers at the bottleneck for long-range spatial coherence.

5.3 Attention Layers for Long-Range Coherence

Diffusion U-Nets also typically incorporate self-attention layers, most commonly at the bottleneck and lower-resolution stages, alongside the standard convolutional blocks Day 33's U-Net used throughout. This addition serves the same purpose Day 33 §8.1's SegFormer discussion identified for segmentation: a convolution's receptive field grows only gradually with depth (Day 29 §3.3), while self-attention can relate distant spatial positions directly, which matters specifically for diffusion because generating a globally coherent image (consistent lighting across a whole scene, a correctly proportioned single object) requires exactly the kind of long-range spatial reasoning that a purely convolutional network, per Day 33 §8.1's argument, has to build up gradually through many layers rather than access directly.

It is worth being concrete about a failure mode this attention addition specifically addresses, since it makes the motivation less abstract. A purely convolutional denoising network, relying only on local receptive fields at any given layer, can end up making locally plausible but globally inconsistent decisions — for instance, generating a face with two eyes of subtly different, individually-plausible colors, because the local receptive field processing one eye has no direct access to information about what the other eye, elsewhere in the image, currently looks like at that stage of denoising. Self-attention layers at the bottleneck give the network direct access to global image context at every one of these decisions, letting it enforce exactly this kind of long-range consistency that purely local convolutional processing struggles to guarantee.

6 Sampling Strategies & Speed

Section 4's training stability is diffusion's central advantage over Day 34's GANs; this section develops diffusion's central practical drawback, and the direct counterpoint Section 9's comparison will weigh against that advantage.

6.1 DDPM's Original Slow Sampling

Generating a single sample requires running Section 3's reverse process through many sequential denoising steps — in the original DDPM (Denoising Diffusion Probabilistic Models) formulation, commonly on the order of a thousand steps, each one a full forward pass through Section 5's U-Net. This stands in stark, direct contrast to Day 34's GAN generator, which produces a complete sample in a single forward pass. Where a trained GAN can generate a batch of images in roughly the time it takes to run one network forward, DDPM-style sampling requires running the network forward roughly a thousand times sequentially for every single sample, illustrated in Figure 6 — a substantial, practically significant speed cost that has motivated most of this section's subsequent developments.

It is worth being concrete about why this cost cannot simply be parallelized away, since a thousand forward passes might otherwise sound like something modern GPU hardware could trivially run all at once. Each step of DDPM's reverse process depends directly on the output of the previous step — x_{t-1} is computed from x_t , which was itself just computed from x_{t+1} — a strictly sequential dependency chain that cannot be parallelized across steps the way a single large batch of independent images can be parallelized across samples. This is a fundamentally different kind of computational cost than, say, processing a larger batch size, which is why DDPM sampling remains slow even on the most capable single-GPU hardware, and why Section 6.2 onward focuses on reducing the number of sequential steps rather than simply throwing more parallel compute at the problem.

6.2 DDIM: A Faster Deterministic Sampling Shortcut

DDIM (Denoising Diffusion Implicit Models) reformulates the sampling procedure to allow skipping steps — rather than necessarily stepping through every single timestep from T down to 1, DDIM's reformulation permits jumping across multiple timesteps at once while still producing a coherent final image, typically reducing sampling to somewhere on the order of tens of steps rather than DDPM's roughly one thousand, a meaningful practical speedup with comparatively modest quality cost. DDIM also makes sampling deterministic given a fixed starting noise (rather than DDPM's inherently stochastic sampling process), a property with a useful side benefit: the same starting noise reliably produces the same output, enabling meaningful latent-space interpolation between two generated images by interpolating their starting noise vectors, similar in spirit to Day 34 §8.2's StyleGAN latent-space manipulation, though arrived at through a different mechanism.

It is worth being clear about what DDIM actually changes versus what it leaves untouched, since it is easy to overstate. DDIM does not retrain the underlying denoising network at all — a model trained exactly per Section 4's noise-prediction objective can be sampled from using either the original DDPM procedure or DDIM's faster, deterministic reformulation, entirely as a choice made at sampling time. This is a genuinely convenient practical property: a single trained model supports multiple sampling strategies with different speed-quality tradeoffs (Section 6.4), without needing to train separate models for each one, unlike Day 34's GANs, where the trained generator's architecture and the resulting speed-quality profile are fixed together at training time.

6.3 Further Speedups: Distillation and Few-Step Samplers

Beyond DDIM's step-skipping, further research has pushed sampling speed considerably further through distillation — training a new, faster student model to directly approximate what many steps of the original, slower model's sampling process would produce, collapsing what previously required dozens of sequential network evaluations into a handful, or in the most aggressive cases, a single one. Section 10.1 returns to this frontier directly, since closing the remaining speed gap with GANs entirely is one of the most actively pursued research directions building on the foundation this section establishes.

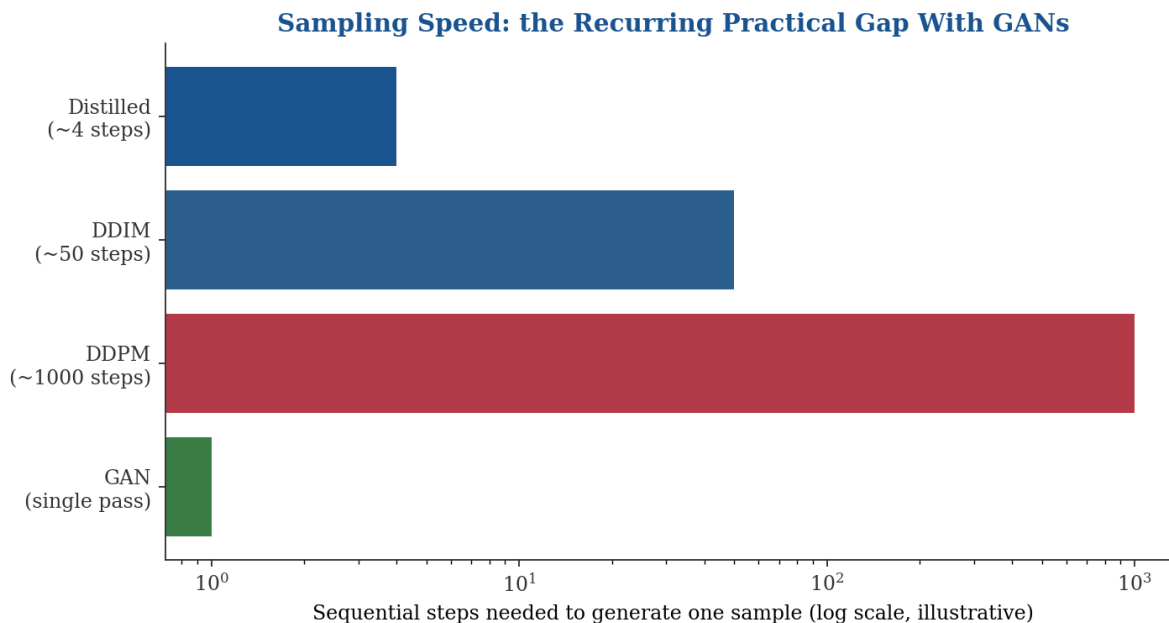


Figure 6. The recurring practical gap this day keeps returning to: a GAN generates a sample in one forward pass; even DDIM's faster sampling needs dozens of sequential steps, and original DDPM sampling needs roughly a thousand — Section 9 weighs this against Section 4's training-stability advantage directly.

6.4 The Speed-Quality Tradeoff

It is worth naming the tradeoff this section's progression embodies explicitly, since it recurs at every step-count choice a practitioner makes: fewer sampling steps generally means faster generation but some cost to sample quality or fine detail, since each individual denoising step is being asked to do more correction at once with less opportunity for the process to gradually refine its way toward a coherent result. Choosing a step count is, in practice, a direct, tunable dial between generation speed and output quality — there is no universally correct number of steps, only a choice appropriate to a given application's latency tolerance and quality requirements, echoing this compendium's now-familiar "no universal winner, only a fit for constraints" framing one level deeper, within diffusion sampling itself rather than only across architecture families.

It is worth being concrete about where this tradeoff typically becomes visible in practice, since "some cost to quality" can otherwise sound abstract. At very low step counts, generated images commonly show soft, under-resolved fine detail (texture that looks slightly smeared rather than sharp) and occasional larger-scale coherence errors (Section 5.3's attention layers have less opportunity across fewer steps to enforce the long-range consistency they are designed for), while at step counts well past the point of diminishing returns, additional steps buy progressively smaller quality improvements for a proportional increase in generation time — meaning the practically useful choice usually sits at a knee in this curve rather than at either extreme, a value worth determining empirically for a given model and application rather than assumed from a published default.

7 Conditional & Guided Generation

Every mechanism Sections 2 through 6 developed produces unconditional generation, exactly as Day 34 §2–§5's baseline GAN did — sampling from the training distribution's overall shape with no way to specify what should be generated. This section develops diffusion's answer to Day 34 §6's conditional-generation question, via a genuinely different mechanism than Day 34's conditional GAN used.

7.1 Classifier Guidance and Classifier-Free Guidance

Classifier guidance steers the reverse process using gradients from a separately trained classifier: at each denoising step, the classifier's gradient with respect to a desired class pushes the denoising trajectory toward images the classifier confidently recognizes as that class, nudging generation toward the requested condition at every step along the way. Classifier-free guidance, developed shortly after, removes the need for a separate classifier network entirely: the same diffusion model is trained both with and without conditioning information (randomly dropping the condition during some training steps), and at sampling time, the difference between the model's conditioned and unconditioned predictions is amplified to push generation more strongly toward the specified condition — an elegant simplification that eliminates an entire second network Section 7.1's original classifier-guidance approach required.

It is worth being precise about why classifier guidance's separate classifier was a genuine practical burden, beyond simply being an extra model to train. That classifier has to work correctly on noisy images at every timestep along the diffusion process, not only on clean images — a standard pretrained classifier (Day 29's subject), trained exclusively on clean images, has no reason to behave sensibly on Section 2's heavily-noised intermediate states, meaning classifier guidance in practice required training a bespoke, noise-aware classifier specifically for this purpose. Classifier-free guidance's insight — that the diffusion model itself, trained both conditionally and unconditionally, already implicitly contains the information needed to steer generation — removed this entire bespoke-classifier requirement at a stroke, which is a large part of why it was adopted so broadly and so quickly once introduced.

Research Spotlight: *Classifier-free guidance is worth appreciating as more than a minor implementation convenience. By removing the need for a separate, independently trained classifier, it removed a genuine practical bottleneck — a classifier trained specifically to provide gradients at every noise level along the diffusion process is itself a nontrivial additional model to build and maintain — while empirically producing better-guided samples than classifier guidance typically achieved. This single technique had an outsized impact on the practical quality of modern text-to-image systems specifically, and is a large part of why text-conditioned diffusion models became reliably steerable at the scale and quality seen in widely used text-to-image systems today.*

7.2 Text-Conditioned Generation and Cross-Attention

Text-conditioned generation — the mechanism behind modern text-to-image systems — extends Section 7.1's conditioning framework with a text prompt as the condition, illustrated in Figure 7. The text is first

passed through a text encoder (often a CLIP-style or similar language encoder, echoing Day 26 §10.1 and Day 33 §8.2's earlier previews of joint text-image representations) to produce a sequence of text embeddings, and these embeddings are injected into Section 5's denoising U-Net via cross-attention layers, in which the U-Net's spatial features attend to the text embeddings at each denoising step — letting every region of the generated image be influenced by whichever words in the prompt are most relevant to it, rather than the text condition being folded in as a single, undifferentiated global vector.

It is worth being explicit about what "attend to the text embeddings" means mechanically, since cross-attention is a specific, well-defined operation rather than a vague conditioning gesture. At each cross-attention layer, every spatial location in the U-Net's current feature map computes a weighted combination of the text embeddings, where the weights are determined by how relevant each word's embedding is to that specific spatial location's current content — a region of the image currently being denoised toward fur texture, for instance, can learn to weight a prompt word like "fox" heavily, while a region toward the image's background can weight a different word more heavily, all within the same single forward pass through the network, at every one of Section 6's sequential denoising steps.

This is worth explicitly connecting back to Day 34 §6's conditional GAN discussion, since both sections solve a structurally similar problem via genuinely different mechanisms. Day 34 §6's cGAN concatenated a condition (typically a simple class label) into both the generator and discriminator's computation at a small number of points. Diffusion's cross-attention conditioning is considerably richer: it operates at every denoising step, and cross-attention specifically allows different spatial regions of the image to attend to different parts of the conditioning signal (different words in a prompt), a meaningfully more expressive conditioning mechanism than Day 34 §6's label concatenation, though built to serve the same fundamental purpose of controllable generation.

8 Latent Diffusion

Sections 2 through 7 have all implicitly assumed the diffusion process runs directly in pixel space — every denoising step operating on a full-resolution image. This section develops a crucial practical modification that made high-resolution diffusion tractable at all outside of very large compute budgets.

8.1 The Motivation: Pixel-Space Diffusion Is Expensive

Section 6 already established that generating a single sample requires many sequential passes through Section 5's U-Net. Running every one of those passes at full pixel resolution — particularly at the high resolutions modern applications expect — is computationally expensive enough that Section 6's already-significant speed cost compounds with a further, separate resolution-driven cost: each individual denoising step itself becomes more expensive as resolution increases, on top of needing many such steps in sequence.

It is worth being specific about how these two costs actually compound rather than simply add, since the practical consequence is worse than either cost considered in isolation. Doubling image resolution roughly quadruples the pixel count a convolutional U-Net has to process at every layer, meaning a single denoising step at high resolution can be many times more expensive than the same step at a lower resolution — and that already-inflated per-step cost is then still multiplied by Section 6.1's roughly-a-thousand sequential steps (or Section 6.2's reduced but still substantial dozens of steps). This multiplicative interaction between resolution cost and step-count cost is precisely what made naive, full pixel-space diffusion impractical at high resolution outside of the largest available compute budgets, and precisely the problem Section 8.2's latent-space insight was designed to solve.

8.2 The Latent Diffusion Insight

Latent diffusion's solution, illustrated in Figure 8, is to run the entire diffusion process — Sections 2 through 7's forward corruption, reverse denoising, and conditioning mechanisms — not in pixel space, but in a much smaller, compressed latent space produced by a separately, pretrained autoencoder. A real image is first compressed by the autoencoder's encoder into a compact latent representation; the diffusion process (training and sampling alike) operates entirely on this compact latent, at a small fraction of the original pixel count; and only at the very end of the sampling process is the final denoised latent passed through the autoencoder's decoder to produce a full-resolution pixel-space image.

It is worth being explicit about why this reordering — compress first, then diffuse, rather than diffuse first at full resolution — is a genuinely different design choice from simply generating a smaller image and upscaling it afterward, since the two can sound superficially similar. A downscaled-then-upscaled image loses fine detail that a generic upscaling algorithm has to hallucinate or interpolate, with no awareness of what the image actually depicts. A well-trained autoencoder's latent space, by contrast, is specifically optimized to preserve the visually important information needed for faithful reconstruction (Section 8.3), meaning the diffusion process operating on that latent is working with a compact but semantically rich representation, not a merely downsampled, detail-poor one — the compression is learned and content-aware, not a generic resolution reduction.

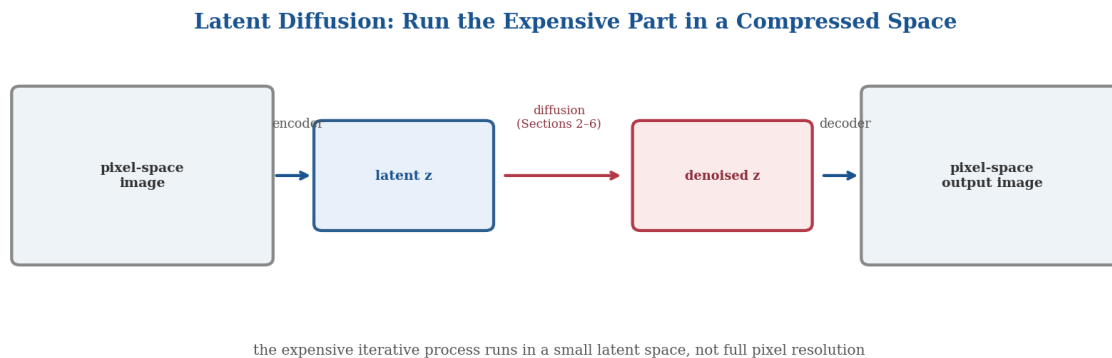


Figure 7. Latent diffusion runs Sections 2–7’s entire diffusion process in a small, pretrained autoencoder’s latent space rather than full pixel resolution — the specific insight that made high-resolution diffusion practical on consumer, single-GPU hardware.

8.3 Why This Made High-Resolution Diffusion Practical

Because the latent space is a small fraction of the original pixel count, every one of Section 6’s many sequential denoising steps becomes correspondingly cheaper — the compute savings compound directly with the step-count reduction Section 6.2 and 6.3 already achieved, rather than competing with it. This combination is specifically what made high-resolution diffusion practical on consumer-grade, single-GPU hardware rather than requiring the large, specialized compute clusters full pixel-space diffusion at high resolution would otherwise demand, and it is worth being explicit that this is not a minor optimization: latent diffusion is the architecture underlying most modern text-to-image systems in practical, widespread use today, not a niche variant alongside a more common pixel-space approach.

It is worth being precise about what the pretrained autoencoder’s own training has to achieve for this to work well, since the whole approach depends on it. The autoencoder must be trained (typically beforehand, independently of the diffusion model itself) to compress an image into a latent representation and reconstruct it with minimal loss of visually important detail — a task structurally related to, though with a different objective than, the encoder-decoder architectures Day 33 §3.2 developed for segmentation, here optimized for faithful reconstruction rather than dense per-pixel classification. A poorly trained autoencoder would introduce compression artifacts into every generated image regardless of how well the diffusion process itself performs in latent space, since the diffusion model’s output quality is fundamentally capped by how much visually relevant information the latent representation actually preserves.

It is worth closing this section with the practical scale of the resulting savings, since the numbers involved are what actually explain latent diffusion’s outsized real-world impact. A typical latent space might compress an image by a factor on the order of tens in each spatial dimension, translating to a reduction in total pixel count of two to three orders of magnitude — meaning the same hardware that could only run pixel-space diffusion at a small, low-resolution image size can run latent diffusion at a considerably larger,

higher-resolution output, simply because the expensive iterative process (Sections 2–7) is operating on a representation with that much less raw data to process at every one of Section 6's many sequential steps.

9 GANs vs. Diffusion: A Direct Comparison

This is the comparison Day 34 §1 and §10.1 both promised, and every section of this day has been building toward it. With Sections 2 through 8 having developed diffusion's mechanics in full, this section places the two paradigms directly side by side.

9.1 Training Stability

Diffusion's advantage here is substantial and unambiguous, following directly from Section 4.2's argument: a single network trained against a simple, fixed regression target (Section 4.1) versus Day 34 §3–§4's two networks locked in an adversarial game with no single reliable progress signal and several well-characterized failure modes. This is diffusion's clearest, least-qualified advantage over GANs.

It is worth being specific about what this stability advantage actually means for a practitioner's day-to-day experience training each kind of model, since the abstract comparison above can otherwise understate its practical weight. A diffusion training run's loss curve, per Section 4.2, behaves like a familiar, well-understood regression loss — falling reliably as training progresses, with no need for Day 34 §3.2's careful joint reading of two interacting curves. A practitioner training a diffusion model from scratch can, in large part, trust the loss curve the way Day 29's classification training let them, where a GAN practitioner has to additionally watch for Day 34 §4's specific failure signatures throughout training regardless of what either loss curve shows.

9.2 Sample Quality and Diversity

Diffusion models typically avoid Day 34 §4.1's mode collapse largely by construction: because Section 4's training objective has no adversarial component pushing the generator toward whatever narrow output currently fools a discriminator, there is no equivalent incentive structure for a diffusion model to collapse onto a limited set of outputs, and diffusion models empirically tend to produce more diverse samples covering more of the true training distribution than GANs commonly achieve. This does not mean diffusion sample quality is unconditionally superior in every respect — Day 34 §8.2's StyleGAN, as Day 34 §10.1 already noted, remains highly competitive on image quality for the specific domains it was designed for — but diffusion's diversity advantage specifically is a consistent, well-documented empirical pattern rather than an occasional exception.

It is worth reading this diversity comparison alongside Day 34 §9.4's precision-recall framing directly, since it gives the qualitative claim above a more precise, measurable shape. In Day 34 §9.4's terms, diffusion models tend to score well on generative recall (covering a broad swath of the true data distribution's

diversity) essentially by default, whereas a GAN's recall specifically depends on successfully avoiding mode collapse during training — meaning the diversity comparison between the two paradigms is not simply "diffusion is better" in the abstract, but a more specific claim that diffusion's recall is more reliably high across training runs, without needing the careful monitoring Day 34 §4.1's mode-collapse diagnostics require.

9.3 Inference Speed

Here the comparison inverts cleanly in the opposite direction, following directly from Section 6: a GAN's single forward pass versus diffusion's many sequential denoising steps (Figure 6), even after DDIM and distillation's speedups (Section 6.2–6.3). This is GAN's clear, unambiguous advantage, mirroring diffusion's equally clear advantage in Section 9.1 — the two paradigms trade their principal strengths almost symmetrically, training stability against generation speed.

It is worth being specific about which applications this speed gap actually rules out for diffusion, rather than treating it as an abstract inconvenience. Interactive, real-time applications — a live camera filter, an on-device generative effect responding to user input within a single frame's latency budget — are the clearest case where Day 34's GAN generator's single forward pass remains close to a hard requirement, and where even Section 10.1's consistency-model research, promising as it is, has not yet fully closed the gap at the time of this writing. Batch or offline generation — producing a set of images ahead of time rather than in direct response to live user interaction — is comparatively unaffected by this gap, which is precisely why most current text-to-image systems (Section 7.2, Section 8.3), operating in this batch-generation mode rather than a hard real-time one, have adopted diffusion despite its speed disadvantage relative to GANs.

9.4 Training Cost and Data Efficiency

Beyond stability, it is worth noting a related but distinct consideration: diffusion models often require substantial training data and compute to reach their best quality, though this cost is offset in practice by their comparative reliability — a diffusion training run is less likely to require the kind of careful, hands-on intervention (Day 34 §3.3's balancing act, Section 4 of Day 34's failure-mode diagnosis) that a GAN training run frequently does, meaning the total practical cost of reaching a working, high-quality model is not simply a matter of comparing raw compute budgets in isolation.

It is worth adding a further data-efficiency consideration specific to diffusion's noise-prediction objective (Section 4.1), since it has a direct practical consequence for smaller-scale reproductions. Because every training example, at every randomly sampled timestep, provides a well-defined regression target regardless of how much of the dataset the model has already seen, diffusion training tends to make efficient, consistent use of a comparatively modest dataset without the specific risk Day 34 §5.4's DCGAN discussion flagged for GAN discriminators — a discriminator overfitting to a small dataset's specific real examples (Day 34's Common Pitfalls checklist) has no direct analogue for a diffusion model's noise-prediction target, since

there is no second network capable of "memorizing" real examples in the adversarial sense Day 34 describes.

9.5 Practical Guidance: When Each Is the Better Choice

Putting Sections 9.1 through 9.4 together: diffusion is generally the better choice when training reliability matters more than generation speed, when sample diversity is a priority, and when the application can tolerate multi-step, non-real-time generation (batch content creation, offline generation pipelines, most modern text-to-image systems per Section 8.3). A GAN remains the better choice when real-time or near-real-time single-pass generation is a hard requirement, when the training instability Day 34 §3–§4 covers can be actively managed by an experienced practitioner, or when a specific, highly-tuned architecture like StyleGAN (Day 34 §8.2) already exists for the exact domain in question. As with every architectural comparison in this compendium since Day 30 §9, there is no universal winner here, only a fit for a given application's constraints.

It is worth adding one further, practical decision-making consideration beyond the direct technical tradeoffs above: available tooling and pretrained resources. Because latent diffusion (Section 8) underlies most current widely-used text-to-image systems, a practitioner starting a new image-generation project today is likely to find considerably more mature pretrained models, fine-tuning tooling, and community documentation available for diffusion-based approaches than for GAN-based ones, independent of the underlying technical tradeoffs Sections 9.1 through 9.4 developed — a genuinely practical consideration worth weighing alongside the technical comparison, since the best-suited architecture in principle is not always the most practical starting point in a given software ecosystem's current state.

9.1.1 Side-by-Side Comparison Table

Synthesizing the full comparison in one place: training stability strongly favors diffusion (Section 9.1); sample diversity favors diffusion, with GANs remaining competitive on raw quality for specialized, well-tuned domains (Section 9.2); inference speed strongly favors GANs (Section 9.3); controllability is roughly comparable in capability though achieved through different mechanisms — Day 34 §6–§7's conditional and image-to-image GANs against this day's Section 7's classifier-free guidance and cross-attention conditioning; and total practical training cost is genuinely mixed, since diffusion's larger raw compute needs are offset by requiring less hands-on intervention to reach a working result (Section 9.4). No single row of this comparison decides the question on its own — the right choice depends on which of these dimensions matters most for a given application.

10 Research Frontiers

10.1 Consistency Models and Closing the Speed Gap

Section 6.3 already introduced distillation as a way to reduce sampling to a handful of steps; consistency models push this idea toward its logical limit, training a model specifically to map any point along the noising trajectory directly to the final clean image in a single step (or a very small, fixed number of steps), rather than requiring the many sequential refinements Section 3's reverse process otherwise needs. This research direction is worth framing explicitly as an attempt to close Section 9.3's speed gap with GANs directly, without abandoning Section 9.1's training-stability advantage that motivated diffusion in the first place — if successful at scale, it would substantially narrow the one dimension (Section 9.3) where GANs currently hold a clear, largely uncontested advantage.

It is worth being precise about what makes this a genuinely difficult research problem rather than a straightforward engineering optimization. A model trained to jump directly from any noisy point to the clean image in one step has to implicitly learn everything Section 3's many-step reverse process would otherwise discover gradually, one small correction at a time — collapsing a long, incremental refinement into a single leap risks losing exactly the fine-grained detail and coherence that many small steps naturally build up. Consistency models address this with a specific training objective enforcing that the model's predictions remain consistent along a given noising trajectory regardless of which point on it they start from, rather than training a one-step model directly against Section 4.1's noise-prediction target alone, which empirically tends to underperform when compressed too aggressively.

10.2 Video Diffusion: A Forward Reference to Day 36

Extending this day's image-generation framework to video — generating a coherent sequence of frames rather than a single image, with consistent motion and object identity across time — is an active research frontier directly connecting to Day 36's video processing content. Video diffusion must solve a genuinely harder version of Section 5.3's long-range coherence problem: not only spatial coherence within a single frame, but temporal coherence across many frames, echoing Day 32's tracking concerns (maintaining consistent object identity across time) from a generative rather than discriminative direction.

It is worth being specific about the additional architectural machinery this typically requires, since it does not simply fall out of running Section 5's image-denoising U-Net independently on each frame. Applying this day's per-image denoising process to every frame of a video independently would produce exactly Day 32 §1's flicker problem in a generative context — each frame individually plausible, but with no guarantee of consistency between adjacent frames, since nothing in the per-frame process has any awareness of neighboring frames. Video diffusion architectures address this by adding temporal attention or 3D convolutional layers that let the denoising process at a given frame draw directly on information from neighboring frames, a generative-context echo of the same temporal-modeling need Day 32 §8's activity recognition architectures addressed for a discriminative purpose.

10.3 3D-Aware Diffusion: A Forward Reference to Day 36

A related frontier extends diffusion to generate or reason about 3D structure directly — producing outputs consistent with an underlying 3D scene rather than a single 2D projection of one, a direct forward reference to Day 36's broader 3D vision content. This connects to the same generative-versus-discriminative distinction Section 10.2 raised: where Day 36's core 3D vision content will largely be discriminative (understanding 3D structure from captured images), 3D-aware diffusion asks the generative question — producing new, plausible 3D-consistent content — extending this day's generative arc into the spatial dimension Day 36 develops for its own, primarily discriminative purposes.

It is worth naming a specific practical difficulty this direction has to solve that neither Section 10.2's video extension nor this day's core image-generation content faces in quite the same form: 3D training data is considerably scarcer than the large 2D image datasets this compendium's earlier days have relied on. Several approaches to 3D-aware generation work around this by training on ordinary 2D images captured from multiple viewpoints and learning an implicit 3D-consistent representation from that multi-view 2D evidence alone, rather than requiring genuine, explicitly labeled 3D training data — a workaround conceptually related to Day 36's own multi-view 3D reconstruction techniques, now applied to a generative rather than reconstructive end.

10.4 Diffusion Beyond Images

Echoing Day 34 §10.3's brief note on GAN applications beyond images, diffusion's underlying framework — learn to reverse a gradual corruption process — is not inherently image-specific either, and has been applied to audio generation and molecular structure generation, among other domains, built on the same forward-corruption/reverse-denoising framework Sections 2 and 3 developed, with domain-specific architectural changes replacing Section 5's convolutional U-Net where the underlying data is not itself a spatial image.

It is worth being specific about what stays constant and what changes when diffusion is applied outside the image domain, since it clarifies exactly how general this day's core framework actually is. Sections 2 through 4's forward process, reverse process, and training objective transfer essentially unchanged — noise still gets added according to a schedule, the network still learns to predict it, and the loss is still a simple regression. What changes is entirely Section 5's backbone architecture: audio diffusion typically replaces the 2D convolutional U-Net with a 1D-convolutional or spectrogram-based variant suited to audio's different data structure, and molecular diffusion replaces it with architectures suited to graph-structured molecular data, but in every case the conceptual core this day developed in Sections 2 through 4 remains the same.

10.5 Ethical Considerations: An Even More Capable Mechanism

Day 34 §10.2 developed this compendium's core generative-ethics discussion in full — synthetic media, deepfakes, consent, and the ongoing detection arms race — and every principle established there applies

here with, if anything, added weight rather than needing to be restated from scratch. Modern text-to-image diffusion systems (Section 7.2, Section 8.3) are, at the time of this writing, generally capable of higher-fidelity, more precisely controllable synthetic image generation than the GAN-based systems Day 34 covered, meaning the same consent and disclosure principles Day 26 §9.3 and Day 34 §10.2 established apply to a mechanism with correspondingly greater capability for both creative benefit and potential misuse — worth carrying forward explicitly rather than assuming Day 34's ethics discussion was somehow specific to GANs alone.

It is worth naming one further consideration specific to diffusion's conditioning mechanism (Section 7) that Day 34's GAN-focused ethics discussion did not need to address in quite the same form: because text-conditioned diffusion systems can be steered with arbitrary natural-language prompts rather than Day 34 §6's fixed, predetermined class labels, the space of what can be requested is vastly larger and harder to fully anticipate or restrict in advance. This makes prompt-level safeguards — filtering or refusing requests for disallowed content before generation even begins — a genuinely necessary complement to Day 34 §10.2's output-level detection and disclosure principles, since the flexibility that makes text-conditioned diffusion so creatively useful is the same flexibility that makes exhaustively anticipating every possible misuse considerably harder than it was for a GAN conditioned on a small, fixed set of labels.

It is worth closing this day, and this module's generative arc, with a direct synthesis rather than treating diffusion as simply "the newer, better option." Day 34 developed one complete, genuinely useful generative paradigm; this day developed a second, structurally different one solving the same problem through a different mechanism, with a different, largely inverse profile of strengths and weaknesses (Section 9). Both remain active, legitimate tools in computer vision practice, and the chain from Day 33's discriminative segmentation through Day 34's adversarial generation and this day's denoising-based generation completes the module's central arc: from mapping images to labels, to producing entirely new images two structurally different ways.

11 Common Pitfalls: A Practical Debugging Checklist

Because this day combines multiple architectural stages and a dense evaluation methodology, a poor result can trace back to any of several places. The following checks catch the most common issues.

- Verify Section 2.2's closed-form sampling is implemented correctly before assuming a training bug elsewhere — an error in directly computing x_t from x_0 at a random timestep will silently corrupt every training example's noise target, producing a model that appears to train (the loss will still decrease) but never learns to denoise correctly.
- Confirm the timestep embedding (§5.2) is actually reaching every part of the U-Net it needs to, not just the input layer — a network that cannot tell which noise level it is looking at will struggle to

produce a single, consistent denoising behavior across the very different noise levels seen early versus late in the reverse process.

- Distinguish a genuine training problem from an expected consequence of too few sampling steps — per §6.4, reducing DDIM's step count trades some quality for speed by design; blurry or artifact-heavy samples from an aggressively low step count are not necessarily evidence the model itself was trained incorrectly.
- Check classifier-free guidance's strength setting (§7.1) before concluding a conditioning mechanism has failed — too low a guidance strength under-steers generation toward the specified condition, while too high a strength can produce oversaturated, artifact-heavy outputs; the right value is a genuine tuning choice, not a fixed default that works everywhere.
- If using latent diffusion (§8), verify the pretrained autoencoder's reconstruction quality independently before debugging the diffusion model itself — per §8.3, generation quality is capped by how much visually relevant detail the latent space preserves, and a poor autoencoder will produce compression artifacts no amount of diffusion-model tuning can fix.
- Verify sampling and training use a consistent noise-prediction versus image-prediction convention throughout — per §4.1, the two targets are mathematically related but not interchangeable at the code level, and mixing conventions between a training script and a sampling script is a silent, easy-to-introduce bug that produces plausible-looking but incorrect noise estimates.
- When comparing DDPM against DDIM sampling (§6.1–§6.2), fix the underlying trained model and vary only the sampling procedure — comparing two different sampling strategies run on two separately trained models conflates the sampling-speed question this section is about with an unrelated question of training-run variance.

11.1 A Note on Reproducibility

Per Day 28 §11.1 through Day 34's reproducibility guidance, diffusion training shares the standard sources of randomness (weight initialization, data augmentation, Day 27 §6) with a diffusion-specific addition: the random timestep sampled per training example (§4.1) and the random noise sampled at that timestep are both further sources of run-to-run variance, and — separately, at sampling time rather than training time — the random starting noise x_T (§3.2) determines which specific image a trained model produces for a given prompt or condition. Fixing all of these seeds explicitly, both during training and when generating comparison samples, is necessary for genuinely reproducible results, echoing Day 34's identical concern for GAN noise-vector sampling but with an additional training-time random-timestep component this day's objective introduces.

11.2 A Note on Comparing Against Published Benchmarks

As with every benchmark note since Day 31 §11.2, comparing a small-scale diffusion implementation's own FID (Day 34 §9.2) or qualitative results directly against published diffusion benchmark figures is rarely meaningful without significant caveats — published results are typically computed with far more training compute, far more sampling steps, and, for latent diffusion specifically (§8), a pretrained autoencoder trained on a very large, separate dataset that a smaller-scale reproduction may not have access to at all. The FID and sample-quality evaluation methodology Day 34 §9 developed applies to diffusion models exactly as it did to GANs, since Section 1 established both paradigms solve the same evaluation-free generative problem — but the specific published numbers from large-scale systems are not a fair comparison point for a modest reproduction.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Markov chain — A process in which each step depends only on the immediately preceding step, not the full history — the structural property underlying the forward diffusion process (Section 2).

Forward diffusion process — A fixed, non-learned Markov chain that gradually adds Gaussian noise to a real image over T steps until it becomes pure noise (Section 2).

Noise schedule — The predetermined sequence controlling how much noise is added at each forward-process step (linear or cosine, among other choices) (Section 2.1).

Reverse process — The learned denoising procedure that starts from pure noise and iteratively removes noise to produce a coherent image (Section 3).

Score function — The gradient of a distribution's log-density with respect to the data; noise prediction is mathematically equivalent to estimating this quantity at each noise level (Section 3.1).

DDPM — Denoising Diffusion Probabilistic Models — the original diffusion formulation, typically requiring on the order of a thousand sequential sampling steps (Section 6.1).

DDIM — Denoising Diffusion Implicit Models — a faster, deterministic sampling reformulation that permits skipping steps, typically reducing sampling to tens of steps (Section 6.2).

Timestep embedding — A vector representation of the current timestep, injected throughout the denoising U-Net so the network knows which noise level it is processing (Section 5.2).

Classifier-free guidance — A conditioning technique that amplifies the difference between a model's conditioned and unconditioned predictions to steer generation, without requiring a separate classifier network (Section 7.1).

Cross-attention conditioning — Injecting a conditioning signal (e.g., text embeddings) into a denoising U-Net by letting spatial features attend to the conditioning tokens at each step (Section 7.2).

Latent diffusion — Running the entire diffusion process in a compressed latent space produced by a pretrained autoencoder, rather than at full pixel resolution (Section 8).

Consistency models — A research direction training a model to map any point on the noising trajectory directly to the clean image in one or very few steps, aiming to close the speed gap with GANs (Section 10.1).

Video diffusion — Extending diffusion generation to coherent video sequences, requiring additional temporal-attention or 3D-convolutional machinery to maintain consistency across frames (Section 10.2).

Chapter Summary: Key Takeaways

- Diffusion models solve generation by learning to reverse a fixed, gradual noising process, rather than Day 34's adversarial two-network game (§1–§3).
- The forward process is fixed and non-learned; its closed-form property lets training sample any timestep directly, which is what makes training practical (§2.2).
- Training reduces to a simple regression loss — predict the noise that was added — with no adversarial dynamics, no second network, and none of Day 34 §4's failure modes (§4).
- The denoising backbone is Day 33's U-Net with two additions: a timestep embedding and attention layers for long-range coherence (§5).
- Diffusion's central practical drawback is sampling speed — many sequential steps versus a GAN's single forward pass — which DDIM and distillation narrow but do not eliminate (§6).
- Classifier-free guidance and cross-attention give diffusion a richer, step-wise conditioning mechanism than Day 34 §6's label-concatenation approach, underlying modern text-to-image systems (§7).
- Latent diffusion runs the expensive iterative process in a compressed autoencoder latent space, which is what made high-resolution diffusion practical outside large compute budgets (§8).
- GANs and diffusion trade their principal strengths almost symmetrically — diffusion wins on training stability and diversity, GANs win on inference speed — with no universal winner (§9).
- Diffusion's synthetic-media ethics concerns are Day 34 §10.2's concerns carried forward, not a new category, but applied to a generally more capable and controllable generation mechanism (§10.5).
- This day completes the module's generative arc opened in Day 34: two structurally different, both legitimate, answers to the same underlying generative problem (§10.5).

End of Day 35 — ready for gap review and expansion into full compendium.

Theory-only day — no assigned practical project. Ready for gap review and expansion into full compendium.